



CCODEL
RAISING A NEW GENERATION OF
GLOBAL LEADERS

Computer Programming I (Java 1)

COS221

Mr. Otavie Okuoyo

Phd (in view Computer Science)

College of Science and Technology



Visit Our Website

ccodel.covenantuniversity.edu.ng



MODULE FOUR: JAVA CONTROL STRUCTURES

UNIT TWO: LOOPS AND BRANCHING

LOOPS AND BRANCHING

Learning Objectives

By the end of this unit, you will be able to:

- **explain the purpose of loops in programming and identify scenarios where loops are applicable.**
- **compare and contrast the syntax and behavior of for, while, and do-while loops in Java.**
- **describe the functions of branching statements such as break, continue, and return, and demonstrate their use within loop structures.**
- **write Java programs that implement loops and branching statements to solve real-world programming problems.**

Learning Outcomes

At the end of this unit, you should be able to:

- **understand the purpose of loops and when to use them.**
- **differentiate between for, while, and do-while loops.**
- **use branching statements like break, continue and return appropriately.**
- **write Java programs that use loops and branching to solve problem.**

Introduction

- In programming, many tasks like processing multiple users' inputs, working through lists of data, or repeating an action until something changes require repeating steps or making decisions.
- Loops repeat actions and branching statements decide what to do next.
- In this unit, you will get to know the different kinds of loops, like *for*, *while*, and *do-while*, and learn how to use branching commands such as *break*, *continue*, and *return* to guide the flow of your program.

What are Loops?

Loops are used to execute a block of code repeatedly based on a condition. Java provides three loop structures:

- 1. for loop**
- 2. while loop**
- 3. do while loop**

The *for* Loop

Best suited for situations where the number of iterations is known. The syntax is:

```
for (initialization; condition; update) {  
    // Code to execute  
}
```

where:

- **initialization**: Sets the starting point.
- **condition**: Loop continues while this is true.
- **update**: Changes the loop variable each iteration.

The *for* Loop - Example 1

```
public class HelloThreeTimes {  
    public static void main(String[] args) {  
        for (int i = 0; i < 3; i++) {  
            System.out.println("Hello");  
        }  
    }  
}
```

The above code prints “*Hello*” three times. The output is:

Hello

Hello

Hello

The *for* Loop - Example 2

```
public class SimpleSum {  
    public static void main(String[] args) {  
        int sum = 0;  
        for (int i = 1; i <= 10; i++) {  
            sum += i;  
        }  
        System.out.println("Sum: " + sum);  
    }  
}
```

The above code uses *for loop* for calculating the sum of all numbers from 1 to 10 and prints the *sum*. The output is:

Sum: 55

The *for* Loop - Example 3

```
public class MultiplicationTable {  
    public static void main(String[] args) {  
        for (int i = 1; i <= 5; i++) {  
            for (int j = 1; j <= 3; j++) {  
                System.out.printf("%d x %d = %d\n", i, j, i * j);  
            }  
            System.out.println(); // spacing  
        }  
    }  
}
```

The above code prints the first 3 values of the multiplication table of 5.
The output is:

$$5 \times 1 = 5$$

$$5 \times 2 = 10$$

$$5 \times 3 = 15$$

The *while* Loop

The *while loop* is used when the number of iterations is not known in advance and depends on a condition. The syntax is:

initialization;

while (*condtion*) {

 // Statements to execute

update;

}

The *while* Loop - Example 1

```
public class WhileExample1 {  
    public static void main(String[] args) {  
        int i = 4;  
        while (i >= 1) {  
            System.out.println(i);  
            i--;  
        }  
    }  
}
```

The program uses a *while loop* to print numbers from 4 to 1 in descending order. It initializes i to 4 and, as long as $i \geq 1$, prints i and decreases it by 1 each time. The output is as shown:

4

3

2

1

The *while* Loop - Example 2

```
public class WhileSum {  
    public static void main(String[] args) {  
        int sum = 0;  
        int i = 1;  
        while (i <= 100) {  
            sum += i;  
            i++;  
        }  
        System.out.println("Total Sum: " + sum);  
    }  
}
```

This code calculates and prints the sum of numbers from 1 to 100 using a *while loop*.

Total Sum: 5050

The *while* Loop - Example 3

```
import java.util.Scanner;

public class WhileInput {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        String input = "";

        while (!input.equals("exit")) {
            System.out.print("Enter something (type 'exit' to quit): ");
            input = scanner.nextLine();
        }

        scanner.close();
        System.out.println("Exited loop.");
    }
}
```

The code uses a *while loop* to keep reading user input until the user types "*exit*", then it ends the loop.

The *do-while* Loop

This is similar to the *while loop*, but it guarantees that the body of the loop executes at least. The syntax is:

initialization;

do {

// Statements to execute

update;

} while (condition);

The *do-while* Loop - Example 1

```
public class DoWhileNumbers {  
    public static void main(String[] args) {  
        int i = 1;  
        do {  
            System.out.print(i + " ");  
            i++;  
        } while (i <= 5);  
    }  
}
```

The *do while loop* prints numbers from 1 to 5. It runs at least once and continues while $i \leq 5$. The output is:

1 2 3 4 5

The *do-while* Loop - Example 2

```
import java.util.Scanner;
public class PositiveNumber {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int number;

        do {
            System.out.print("Enter a positive number: ");
            number = sc.nextInt();
        } while (number <= 0);

        System.out.println("You entered: " + number);
        sc.close();
    }
}
```

This program repeatedly prompts for a positive number until the user enters a positive number, then displays it.

Branching Statements

Branching statements alter the normal flow of control in loops and methods. They are used to:

1. Exit loops prematurely (*break*).
2. Skip the current iteration and proceed to the next (*continue*).
3. Exit from a method early (*return*).

The break Statement - Example 1

The *break* statement is used to:

- Exit a *switch* statement after executing a specific case, preventing fall-through to the next case

```
public class SwitchBreakExample {  
    public static void main(String[] args) {  
        int day = 3;  
  
        switch (day) {  
            case 1:  
                System.out.println("Monday");  
                break;  
            case 2:  
                System.out.println("Tuesday");  
                break;  
            case 3:  
                System.out.println("Wednesday");  
                break;  
            default:  
                System.out.println("Other day");  
        }  
    }  
}
```

The break Statement - Example 2

The *break* statement is also used to exit a loop (*for*, *while*, or *do-while*) prematurely, even if the loop condition remains true.

```
public class LoopBreakExample {  
    public static void main(String[] args) {  
        for (int i = 1; i <= 10; i++) {  
            if (i == 5) {  
                break; // exit the loop when i equals 5  
            }  
            System.out.println("i = " + i);  
        }  
    }  
}
```

Output:

i = 1

i = 2

i = 3

i = 4

The *continue* Statement

The *continue* statement is used to:

- Skip the current iteration of a loop.
- Immediately jump to the next iteration, bypassing the remaining code inside the loop for the current cycle.

This is useful when certain conditions are met and you want to ignore the rest of the loop body without exiting the loop entirely.

The *continue* Statement - Example 1

```
public class ContinueInForLoop {  
    public static void main(String[] args) {  
        for (int i = 1; i <= 4; i++) {  
            if (i == 3) {  
                continue; // Skip iteration when i is 3  
            }  
            System.out.println("i = " + i);  
        }  
    }  
}
```

The output is:

The code uses a *for loop* to print numbers from 1 to 4, but skips printing when i is 3 using the *continue* statement.

i = 1

i = 2

i = 4

The *return* Statements

The *return* statement is used to:

- Exit a method immediately.
- Optionally return a value from the method to the caller (required for non-void methods).
- Improve branching logic, especially in conditional flows.
- Facilitate error handling or early exits in case of invalid input.

The *return* Statement - Example 1

```
public class ReturnVoidExample {  
  
    public static void greet(String name) {  
        if (name == null || name.isEmpty()) {  
            System.out.println("Invalid name.");  
            return; // Exit early if input is invalid  
        }  
  
        System.out.println("Hello, " + name + "!");  
    }  
  
    public static void main(String[] args) {  
        greet("Jumoke"); // Valid input  
        greet("");       // Invalid input  
    }  
}
```

The output of the code is:

Hello, Jumoke!

Invalid name.

return; is used to exit the method early without continuing further execution.

The *return* Statement - Example 2

```
public class ReturnValueExample {  
  
    public static int square(int number) {  
        return number * number;  
        // Return the result of the calculation  
    }  
  
    public static void main(String[] args) {  
        int result = square(5);  
        System.out.println("Square of 5 is: " + result);  
    }  
}
```

The program defines a method called *square()* to calculate the square of a number and prints the square of 5 (which is 25).

In-Text Questions (ITQs)

1. How does the *continue* statement affect loop execution?
2. Why is understanding loops and branching statements important in Java programming?

In-Text Answers (ITAs)

1. The ***continue*** statement skips the current iteration of the loop and proceeds to the next iteration, effectively bypassing any code after it within the loop body for that cycle.
2. Understanding loops and branching statements is crucial because they allow programs to repeat actions and make decisions dynamically, enabling efficient handling of complex logic and data processing.

Conclusion

- **Mastering loops and branching statements is crucial for writing efficient and flexible Java programs. Loops let you repeat code without rewriting it, while branching lets you make decisions and control the flow based on conditions.**
- **Together, they help you handle tasks like processing multiple inputs, navigating data, and implementing logic that adapts to different situations. By practicing these concepts, you'll be able to build smarter, more dynamic applications that solve real-world problems effectively.**

Summary

- This unit covered Java loops and branching.
- The loops (which are structures used to repeat actions) include: **for loop** (used when iteration count is known); **while loop** (runs while a condition is true) and **do-while loop** (runs at least once, then checks condition).
- We also learned branching statements (that control the flow within loops and methods) such as: **break** (exit loops/switch); **continue** (skip to next loop cycle) and **return** (exit method).
- These help write flexible, efficient programs.